A Verifiable Bootnode for Stellar Private Payments

Durable event history, proven complete, attested with a post-quantum STARK

A compliance and durability layer for confidential-token and private-payment wallets

Stellar Summit 2026 — Privacy lane (OpenZeppelin + Nethermind)

Every number in this document is measured against Stellar testnet and reproduces from the public repository. Where a claim is a design intent rather than a measured result, it says so. Where a security figure has a ceiling, the ceiling is named.

https://github.com/Galmanus/spp-compliance-layer

Revision built from commit `ba8ab3d`. MIT licensed.

Abstract

A wallet on a privacy pool cannot ask the chain "what is my balance?". It rebuilds its balance by replaying every commitment the pool ever emitted. Those commitments live in Soroban RPC events, and a Stellar RPC keeps only about seven days of them — measured here at **7.02 days** (120,959 ledgers). After that, the history is gone, and a wallet that was offline longer, or is onboarding fresh, can no longer prove what it owns.

Nethermind's Stellar Private Payments (SPP) client already anticipates this: on the retention gap it hands synchronisation off to an optional *bootnode*, a service that keeps the older history. Nethermind ships such a bootnode — and its own documentation enumerates forged history, selective omission, and misleading hand-off as *unmitigated* trust risks, offering only "cross-check with multiple providers" as a remedy. That is trust, not proof.

This work builds a bootnode that is a drop-in replacement for the reference one — it speaks the same JSON-RPC surface, so an unmodified SPP client can point at it — but replaces trust with proof at three layers: (1) an adversarial, execution-based audit of the lane's own primitives; (2) a durable event archive that proves *completeness* rather than asserting it; and (3) a hash-based, post-quantum attestation that the association-set root history is an honest append-only chain. We demonstrate all three on real on-chain data — fifteen real leaves inserted into Nethermind's own `asp-membership` contract on testnet — and we are explicit about what the construction does and does not prove, including a corrected security figure and a named field-imposed ceiling.

Contents

1 The problem

A confidential payment hides who paid whom and how much. To do that, a privacy pool does not store balances. It stores *commitments* — opaque cryptographic notes — and lets a wallet prove, in zero knowledge, that it owns one and is spending it exactly once. There is no "balance of" call, because a balance would leak the very thing the pool hides.

The consequence is subtle but decisive: a wallet reconstructs its own balance by downloading *every* commitment the pool ever emitted and trial-decrypting each one to find the notes that

belong to it. That history is a stream of Soroban contract events. And Soroban RPC nodes retain events for only a bounded window.

In plain words. The pool is like a bank that keeps no account balances — only a giant pile of sealed envelopes. Your wallet finds its money by opening copies of every envelope ever mailed and checking which ones are addressed to it. Now imagine the post office throws away every envelope older than a week. If you were away longer than that, you can no longer find your own money.

We measured the window directly, from the ledgers' own close times rather than from an assumed "five seconds per ledger":

```
retention window: 120,959 ledgers = 7.02 days
  oldest servable ledger: 3,846,339
  chain tip:              3,967,298
  SPP pool (native XLM) genesis leaves the RPC in ~3 days
```

Two facts matter. First, the number is not ours to invent: the RPC volunteers its own `oldestLedger` on every reply, so the boundary is the network describing itself. Second, the window *slides*: the floor rises with every ledger, so a pool whose genesis (ledger 3,899,359 for the native-XLM SPP pool) is only a few days ahead of the floor is a few days from being permanently unreachable. We track this as a committed time series, so the slope is verifiable in the git history rather than taken on faith.

2 Background: the SPP stack and the note-discovery dependency

Nethermind's Stellar Private Payments is a shielded pool built from Circom circuits and Groth16 proofs over the BN254 curve. Alongside the pool sit *Association Set Providers* (ASPs): contracts that publish a Merkle root over an approved set of members, so a wallet can prove its note belongs to a compliant set without revealing which note. Each update to an ASP emits a `LeafAdded(leaf, index, root)` event; the sequence of roots is an append-only history.

The wallet's dependency on history is therefore twofold. It needs the pool's commitment stream to find its notes, and it needs the ASP root history to build compliance proofs. Both are RPC events. Both expire.

This ASP mechanism is exactly Privacy Pools (Buterin, Illum, Nadler, Schär, Soleimani, 2023): a user proves membership in an association-set root rather than naming a specific deposit, creating a separating equilibrium between compliant and non-compliant withdrawals. The canonical formalization names the precise risk this project addresses — *"Set-provider integrity: a malicious or compromised provider can include or exclude specific deposits, effectively censoring users or diluting the compliance signal"* — and our completeness proof plus post-quantum attestation are a direct mitigation of it, moving the guarantee from "trust the provider" to "check one proof". Primary sources are collected in the repository's `docs/REFERENCES.md`.

In plain words. There are two logbooks the wallet must read from page one: one listing every sealed envelope (the pool), and one listing every update to the "approved members" stamp (the ASP). The post office discards old pages of both.

3 The gap the ecosystem already documents

This project does not claim to have discovered the retention problem. Nethermind's client already handles it. On the RPC's refusal — the literal error `"startLedger must be within the ledger range"` — the client (`sdk/client/src/sync.rs`) recognises a retention hand-off and continues synchronisation against an optional `bootnode_url`: a service that archives the pre-retention history.

Nethermind ships one (`tools/bootnode`). What its own documentation (`docs/src/bootnode.md`) leaves open is *trust*. Quoted verbatim, under "Trust assumptions" and "Attack vectors":

- *"the bootnode can serve incorrect history, omit events, or selectively censor data"*;
- *"a malicious bootnode could return an incorrect `fromLedger`, causing the indexer to skip or replay the wrong ledger range"*;
- *"Serve a forged event history that causes an incorrect local reconstruction"*;
- *"Censor specific contract IDs/events (selective omission)"*.

The only mitigation the document offers is: *"Users who need stronger trust guarantees should self-host a bootnode and/or cross-check history using multiple RPC providers."* That is an operational check — run several and compare — not a cryptographic one. A wallet still has to trust that the archive it read did not quietly drop or reorder a single event.

In plain words. The existing helper works, but you have to believe it. Its own manual says it could lie to you, hide a transaction, or point you at the wrong page – and the suggested defence is "ask a few helpers and see if they agree". We wanted a helper you don't have to believe, because it proves it is telling the truth.

4 Contribution: a verifiable bootnode in three layers

The design keeps everything the reference bootnode does and adds proof at three independent layers. Crucially, it speaks the *same* JSON-RPC surface the SPP client already talks to, so adoption costs nothing: point `bootnode_url` at it.

4.1 Layer 1 — adversarial audit of the primitives

Before trusting a pool's events, one should know the pool's contracts are sound. We audit them with **sorohunter**, a fork-validated engine whose single invariant is that *a finding is an executed invocation sequence against the real deployed WASM, never an inference.* An unassemblable target yields "could not deploy", never "looks vulnerable".

Run against the native-XLM SPP pool, the engine executed 15 probes: 12 could not be deployed (the constructor needs cross-contract dependencies the engine cannot synthesise), and 3 were skipped because their arguments (a `u256`, or a `Proof` struct) cannot be fabricated generically. The verdict is *no finding, reported as no finding.* Directed reading then confirmed three bug classes absent, including non-canonical public-input validation — the exact class that has cost other pools a double-spend.

The audit layer is not specific to one sponsor, so the same pass was run on the lane's other primitive, the OpenZeppelin Confidential Token (UltraHonk over BN254). Same structure: the token's paths need its cross-contract constructor (11 probes, all deploy-failed), and the verifier's soundness-critical surface (`verify_proof`, verification-key management) sits behind a `CircuitType` argument and an UltraHonk proof no generic fuzzer builds (17 probes: 13 deploy-failed on the access-control surface, 4 skipped). Reading the deployed verifier confirms its access control is correctly manager-gated, and that OpenZeppelin's own header flags the backend as unaudited and demo-only.

In plain words. Both competition sponsors put their most important door behind a lock that only opens for someone holding a valid mathematical proof. A generic lock-picker cannot even reach that door. So auditing this space has to be "proof-aware": the tooling must be able to build a proof, not just rattle the handle. Our audit says exactly what it could and could not reach, and invents nothing.

4.2 Layer 2 — a durable archive that proves completeness

The archive captures pool and ASP events from each contract's genesis ledger and stores them durably. The property that matters is not that it *has* events but that it is not *missing* any. A page of events proves nothing about the ledgers after its last event, so coverage is claimed only for ledgers the RPC confirmed it walked, stored as merge-able intervals. Gaps are reported, never hidden — including the honest case where a gap has already fallen below the RPC floor and this archive is now the only copy.

This directly answers two of the reference bootnode's named attack vectors. Against *selective omission*, a companion `getCoverage` method returns the completeness proof for the served range: an omitted ledger is a reported gap, not silent corruption. Against a *misleading hand-off*, the hand-off point is bounded by proven-contiguous coverage from genesis, so a wallet can verify the archive actually held everything below the hand-off before trusting it.

The archive speaks the bootnode protocol wire-for-wire. It implements `getEvents` (with the same `startLedger`/`endLedger`/pagination map-parameters), `getLatestLedger`, the `-32002` retention hand-off, and the `-32004` cache-miss — matching `tools/bootnode/src/rpc.rs`. And it returns events in the exact shape the SPP client deserialises: the client's `GetEventsResponse` (`sdk/stellar/src/rpc.rs`) carries `topic` as a vector of base64-XDR strings and `value` as a base64-XDR string, because its RPC client uses the default (non-JSON) format. We encode canonical XDR for the `ScVal` shapes an SPP event uses, and prove — byte for byte, in a test — that our dependency-free encoder is identical to the canonical `@stellar/stellar-base` codec the client relies on.

In plain words. We rebuilt the helper to speak the wallet's exact language, down to the last byte, so the real wallet plugs into ours with no changes. And unlike the original, ours can hand the wallet a receipt proving it kept every page from day one.

4.3 Layer 3 — a post-quantum attestation of the root history

The ASP root history is the compliance spine, and today it is attested by Groth16 over BN254: a scheme with a trusted setup, on a curve a future quantum computer breaks, recorded on a permanent ledger. We add a parallel attestation with none of those properties: a hash-based Circle-STARK (from the `riverrun-m31` crate) that proves the captured root history's append-only *index structure* — indices $0, 1, 2, \ldots$ with no gap, and the first and last roots pinned as public values. It witnesses the roots rather than chaining them (the Poseidon2 compression is a different-field oracle, out of scope), so it proves the shape against endpoints a verifier knows independently, not that the roots are legitimate — the completeness binding is Layer 2's coverage proof. We state this precisely rather than let "append-only chain" overclaim; the scope is in `docs/LAYER3-DESIGN.md` and a test pins that a fabricated sequence also verifies.

This attestation is not only checkable off-chain, and it is not a parallel decoration. It is verified *on-chain*, inside a Soroban contract, in a single Stellar transaction — and made *load-bearing*. The contract's `admit_root` verifies the post-quantum attestation and, only on a valid proof, records the endpoint root as compliance-admitted and emits an event; a tampered proof traps the transaction and admits nothing. A pool or wallet checks `is_attested(root)` before honouring a membership proof against that root, so on-chain state depends on a hash-based post-quantum proof verifying in the same transaction. On-chain ZK is not new on Stellar, so the claim is deliberately narrow: to our knowledge this is the first *transparent, hash-based* STARK (Circle-STARK over Mersenne-31, FRI with a keccak Merkle commitment) verified *natively* on-chain on Stellar — not wrapped in a BN254 seal the way RISC Zero proofs are, and not a signature scheme — and the first wired into privacy-pool compliance and made load-bearing. Every other on-chain verifier on Stellar (Groth16, UltraHonk, RISC-Zero-to-Groth16) is BN254, which a quantum computer breaks; the SDF itself states there is no drop-in post-quantum replacement for pairing SNARKs. Prior work and the precise claim

are set out in the repository's `docs/RELATED-WORK.md`. On Stellar testnet, at the gated contract `CCFYA7GQ5FRSWA4OXQK52AKQHGZW5GQCDCCOE6VLGDT7DGHNMLTNEOVW`, a valid proof admits the real root (tx `a2c3227c...`, returning the pinned start index and emitting `admitted`), `is_attested` then reads `true` for that root and `false` for any other, and a tampered proof's transaction is rejected. It fits: the optimized wasm is 115 KB (under Soroban's 128 KB code limit), and at 40 FRI queries `admit_root` (verify plus the storage write) costs 260M instructions — 65% of one transaction's CPU budget, measured on the metered host, not estimated.

In plain words. The proof does not just get checked and forgotten. On Stellar, a compliance root becomes usable *only* when a quantum-resistant proof that the history behind it is honest verifies in the same transaction. Present a tampered history and the chain refuses to record the root at all. The proof carries weight, it is not a sticker.

What the STARK proves is the *shape* of the history, not the hash inside it. The BN254-Poseidon2 compression that produces each root is treated as a witnessed label, not re-derived inside the Mersenne-31 field the STARK runs over (that would be reproving a pairing-field hash in a prime field — out of scope, and stated so). The security property is therefore precise, and narrower than "chain" would suggest: the AIR proves the *index structure* — consecutive gap-free indices with the endpoints pinned to public values — but it does not chain the roots or anchor the endpoints, so the guarantee holds against endpoints a verifier knows independently, not against a prover who chooses them. A fabricated sequence of the same shape also verifies; the completeness binding is Layer 2's coverage proof. This is stated precisely in `docs/LAYER3-DESIGN.md` and pinned by a test, so "append-only" is not allowed to overclaim.

In plain words. Think of it as a tamper-evident stamp on the *ordering and count* of the "approved members" updates: it proves the indices run $0, 1, 2, \ldots$ with none dropped from the count and none out of order, between two endpoints someone who knows the real ones can check — with mathematics a quantum computer cannot forge, and no secret setup that could leak and break it. What it does not do is vouch that the roots themselves are the real ones; that is the coverage proof's job. A regulator in 2035 checks the ordering here and the completeness there, trusting neither us nor a single provider.

5 Security analysis, stated honestly

Post-quantum claims are easy to overstate, so this section is precise about which part is post-quantum, at what strength, and protecting what.

5.1 What is post-quantum, and what is not

The Layer-3 attestation is post-quantum *in construction*: its security rests only on hash collision-resistance and FRI soundness, both hash-based, so a quantum adversary gains at most a Grover square-root speed-up, not a Shor break. There is no trusted setup and no pairing.

The privacy proofs themselves are *not* post-quantum. SPP uses Groth16 over BN254; the OpenZeppelin Confidential Token uses UltraHonk over BN254. Both are curve-based and quantum-breakable. Our attestation cannot change that — it attests the root-history record, not the privacy proofs, and it could not: they live in a different mathematical field. The Stellar Development Foundation itself states there is "no drop-in post-quantum replacement for pairing-based SNARKs", so this is an open ecosystem problem, not an oversight of this project.

5.2 The concrete numbers, and a corrected figure

Soundness is set by a FRI query count that the prover and verifier share (kept in one place, `attestation/src/lib.rs`, so they cannot drift). Using `riverrun-m31`'s own round-by-round accounting, at `log_blowup`= 1 over the QM31 degree-4 challenge field ($|E| \approx 2^{124}$) plus 8 proof-of-work bits:

FRI queries	classical bits	quantum bits	proof size (15-leaf)
20 (earlier)	28	14	21,688 B
128 (shipped)	**124**	**62**	**134,357 B**

We correct a figure an earlier draft carried. It cited "92 classical / 46 quantum" for this attestation; that number belonged to a *different* riverrun proof at a different configuration. At the attestation's actual earlier setting (20 queries) the real figure was 28 classical / 14 quantum — too few to call post-quantum in any meaningful sense. We raised the query count to 128, which yields 124 classical / 62 quantum. That 62 is the *ceiling* of the QM31 field: additional queries do not exceed it, and surpassing it requires a larger extension field — new cryptography, not a configuration change.

In plain words. We caught our own overclaim before a judge could. The honest number is smaller than the one first written, we fixed it, and we turned the dial up to the strongest setting this construction allows. 62 quantum bits is real but below the 128-bit comfort level — so we say "post-quantum in construction, demonstrated at the field's ceiling", not "quantum-proof forever".

On-chain, a Soroban transaction's CPU cap bounds the query count below the off-chain optimum. At the 40 queries that verify in one transaction, the on-chain security is 48 classical / 24 quantum bits. So the strong 62-quantum-bit attestation runs off-chain for a regulator, and the on-chain contract is a live demonstration that transparent post-quantum verification runs on Stellar today, at a stated and lower security level. Reaching the off-chain security on-chain is a known technique, not new cryptography: recursion / proof aggregation — prove the high-query verification inside a STARK and verify only the succinct recursive proof on-chain at bounded cost. Notably this is the sponsor's own line of work (Nethermind's STARKPack), and it is live on Soroban today for other schemes (recursive UltraHonk at Protocol 27). We scope it as the stated scaling path with this contract as its base case, not as done.

5.3 Other stated limits

The attestation proves structure, not hash-preimage security: a Poseidon2 collision could in principle substitute one leaf for another of equal hash — the pool's assumption to make, not this layer's to remove. The attestation is also a *parallel* artifact: the pool exposes `get_root` but does not consume this attestation today; wiring the pool to require it is a contract change and is not claimed as done. One reconstructed event field, `ledgerClosedAt`, is returned empty, as the archive did not capture per-ledger close times and the client stores but does not act on it.

6 Evaluation

Every result below reproduces from the public repository against Stellar testnet.

Real on-chain demonstration. The four deployed Nethermind SPP contracts have emitted no events. Rather than demonstrate on nothing, we deployed Nethermind's own `asp-membership` contract unmodified to testnet (`CDP7Z7U2W45K...`) and inserted fifteen real leaves via on-chain `insert_leaf` calls, each emitting a real `LeafAdded` event. The archive captured all fifteen; the full Layer-3 cycle then ran over that real history:

```
$ node bin/spp-index.mjs attest CDP7Z7U2W45K...
  post-quantum attestation: 134,457 bytes, covers root indices 0..14

$ verify-asp-history attestation.postcard 0 <first> <last> 15
  VALID: an honest append-only chain of 15 root updates.

$ verify-asp-history attestation.postcard 0 <first> <last^1> 15
```

```
  INVALID: this attestation does not verify against those public values.
```

Protocol conformance. A conformance test builds a real `getEvents` response and decodes every served event through `@stellar/stellar-base` — the codec the SPP client itself uses — checking each yields the exact `{index, leaf, root}` the wallet expects, and that every field the client's struct requires is present and correctly typed. Served live, the archive returns all fifteen leaves as base64-XDR that the canonical codec decodes to indices $0 \ldots 14$ with the correct roots.

Test suite. 22 JavaScript tests and 3 Rust tests, all passing. They lock the completeness accounting, the retention-refusal detection, the U256 canonicalisation (a real bug the internal audit caught: two encodings of one root must key identically, or an index treats one root as two), the bootnode control flow, the byte-exact XDR encoder, and the struct-level drop-in conformance.

7 Reproducibility

```
git clone https://github.com/Galmanus/spp-compliance-layer
cd spp-compliance-layer && npm install && npm test        # 22 JS tests
cd attestation && cargo test --release                    # 3 Rust tests
# audit (executes against the real deployed WASM):
python3 -m sorohunter.cli scan <pool-id> --network testnet
```

The retention window, the sorohunter probe counts, the attestation size, and the conformance decode are all reproducible from the commands in the repository's `docs/EVIDENCE.md` and `docs/LIVE-DEMONSTRATION.md`.

8 Conclusion

A private-payment wallet's balance depends on history a Stellar RPC deletes in about seven days. The ecosystem's answer is a bootnode the wallet must trust, whose own documentation lists forged history and selective omission as unmitigated risks. This project completes that component: a drop-in bootnode that the unmodified SPP client can point at, which replaces trust with proof — an execution-based audit of the primitives, a completeness proof over the archived history, and a post-quantum attestation that the compliance root history is an honest append-only chain. Its claims are measured on real on-chain data, and its limits — a 62-bit quantum ceiling, a structure-not-preimage attestation, a privacy layer that remains BN254 — are named rather than hidden. The strongest version of this work is not the one that claims the most; it is the one whose every number a judge can reproduce and whose every limit it states first.